CASE Study: US Bank Analysis (using SQL)

Submitted by: Kuldeep P Agarwala **Batch: DSML Jun26 Beginner**

Objective of the Project:

As a newly onboarded Data Analyst at US Bank, the objective of this project is to answer business questions across five areas — Customer Behavior, Branch Performance, Financial Risk & Revenue, Customer Service, and Card Portfolio — using SQL, and translate the resulting numbers into actionable insights and recommendations for the bank.

Tool used: BigQuery

Data limitations:

- transactions has no card_id — for an account holding cards of different types (e.g. 1 debit + 1 credit), a transaction cannot be attributed to one specific card.
- balance reflects the linked deposit account, not true credit card debt — the schema has no real "amount owed" field for credit products.
- card_number appears to have loaded as a numeric type before being cast to STRING — any leading zero in a real card number would already be lost at ingestion.
- Missing opened_date and date_of_birth values mean some customers/accounts cannot be placed into date- or age-based buckets — handled via NULL/"Unknown" buckets, not fixable with better SQL.

Key Assumptions:

- "Total balance"/"total loans" metrics sum across a customer's or branch's multiple accounts/loans unless a question explicitly says active-only (stated per question).
- NULL dates are excluded where they'd need bucketing (III.A's opened_date), but customers with missing date_of_birth are kept as a separate 'Unknown' age band rather than dropped (II.C).
- Ties in "top N" rankings use DENSE_RANK (all ties included) except III.A, which uses ROW_NUMBER. branch_id is used as a tie-breaker for a strict "exactly 2" reading.

I. A Data types for columns in the 'customers' table

```
SELECT  column_name,
        data_type
FROM    scaler-dsml-sql-kuldeep.us_bank.INFORMATION_SCHEMA.COLUMNS
WHERE   table_name = 'customers'
```

Output

Row	column_name	data_type
1	customer_id	INT64
2	first_name	STRING
3	last_name	STRING
4	email	STRING
5	phone	STRING
6	address	STRING
7	date_of_birth	DATE
8	ssn	STRING
9	created_at	TIMESTAMP

Observation

date_of_birth loaded AS a DATE datatype, created_at AS TIMESTAMP, not AS plain STRING like mentioned in the cASe study's data notes.

Interpretation

While uploading the data to BigQuery, schema auto-detection feature correctly recognised these columns AS DATE and TIMESTAMP instead of STRING.

Recommendation

Always running this data-type check first is a good practice on new datASet. It tells us upfront about the requirement of datatype conversion.

I. B. i What is the earliest and latest customer signup date on record?

```
SELECT  MIN(created_at) AS earliest_account_creation_date,
        MAX(created_at) AS latest_account_creation_date
FROM    scaler-dsml-sql-kuldeep.us_bank.customers
```

Output

Row	earliest_account_creation_date	latest_account_creation_date
1	2015-03-09 00:59:30 UTC	2025-12-13 16:04:35 UTC

Observation

Customer records span from March 2015 to December 2025 (about 10.5 years)

Interpretation

The datASet gives a long period to analyze changes in customer acquisition, retention and churn.

Recommendation

Such datASet would be helpful to perform year-wise and month-wise data like customer acquisition trends, growth periods, and potential opportunities to improve customer onboarding.

I. B. ii What is the earliest and latest transaction date on record?

```
SELECT  MIN(transaction_date) AS earliest_transaction_date,
        MAX(transaction_date) AS latest_transaction_date
FROM    scaler-dsml-sql-kuldeep.us_bank.transactions
```

Output

Row	earliest_transaction_date	latest_transaction_date
1	2020-01-02 06:04:21 UTC	2025-12-29 10:16:09 UTC

Observation

Transaction data starts in January 2020, even though customer records go back to 2015 - a gap of 5-year gap.

Interpretation

Either transaction-level data capture only began in 2020, or those customers were not transacting digitally before 2020. In either situation, we cannot comment on customer's transaction pattern/activity before 2020.

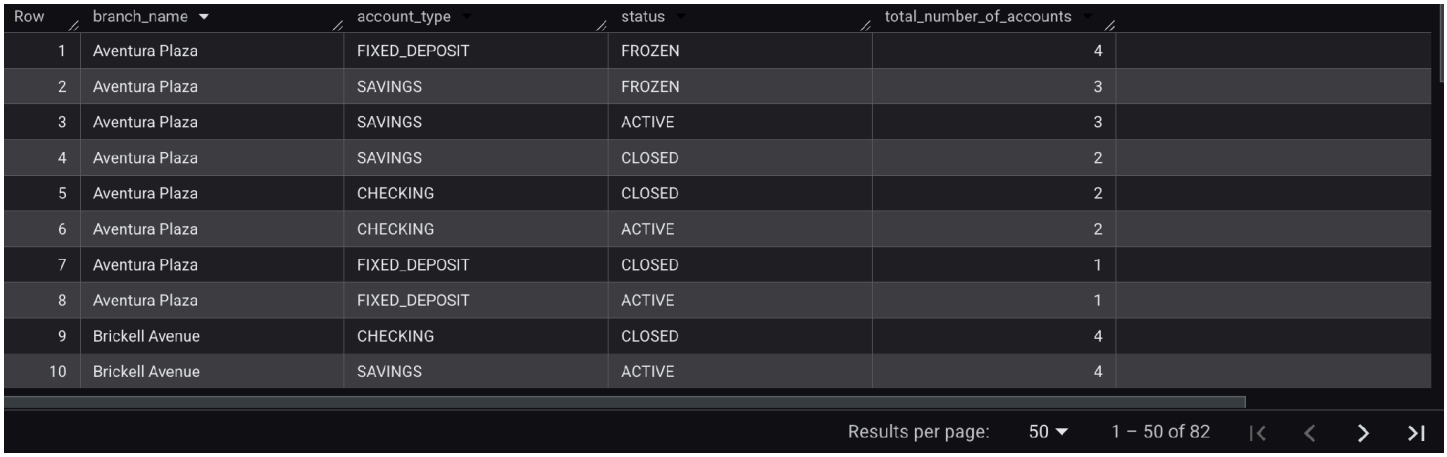
Recommendation

This flags that whenever transaction table data is combined with other data, they would not share equal history.

I. C For every combination of (branch, account type, status), how many accounts exist? Sort by branch name AScending, then account count descending.

```
SELECT      br.branch_name,
            acc.account_type,
            acc.status,
COUNT       (acc.account_id) AS total_number_of_accounts
-- Using COUNT(account_id) instead of COUNT(*) counts the absence of data AS 0 rather than
1
FROM        scaler-dsml-sql-kuldeep.us_bank.branches AS br
LEFT JOIN   scaler-dsml-sql-kuldeep.us_bank.accounts AS acc
ON          br.branch_id = acc.branch_id
GROUP BY    br.branch_name, acc.account_type, acc.status
ORDER BY    br.branch_name, total_number_of_accounts DESC
```

Output (First 10 of 82 rows)



Row	branch_name	account_type	status	total_number_of_accounts
1	Aventura Plaza	FIXED_DEPOSIT	FROZEN	4
2	Aventura Plaza	SAVINGS	FROZEN	3
3	Aventura Plaza	SAVINGS	ACTIVE	3
4	Aventura Plaza	SAVINGS	CLOSED	2
5	Aventura Plaza	CHECKING	CLOSED	2
6	Aventura Plaza	CHECKING	ACTIVE	2
7	Aventura Plaza	FIXED_DEPOSIT	CLOSED	1
8	Aventura Plaza	FIXED_DEPOSIT	ACTIVE	1
9	Brickell Avenue	CHECKING	CLOSED	4
10	Brickell Avenue	SAVINGS	ACTIVE	4

Observation

The output shows that in Aventura Plaza branch, out of the 18 total accounts 7 are FROZEN, 6 are ACTIVE and 5 are CLOSED. With the full 82 row data, we can see that Downtown Miami has the most number of accounts i.e., 28 but it also has the most number of FROZEN accounts i.e., 15.

Interpretation

More than half of Downtown Miami's accounts are FROZEN which could suggest operational gaps/security issues/technical failures etc. This needs to be compared using other metrics as well for confirmation.

Recommendation

A root-cause review of Downtown Miami's frozen accounts needs to be done. Use stronger data tracking tools to find fake profiles, improve account opening steps to stop bad applications early, and immediately report all suspicious accounts to keep the bank safe

II. A Find the average number of transactions for each customer balance category (High Balance >50K, Medium Balance 10K-50K, Low Balance <=10K), along with the number of customers and average balance in each category, sorted by average transaction count from highest to lowest.

```
WITH count_of_txn AS (
  SELECT    account_id,
            COUNT(*) AS txn_count
  FROM      scaler-dsml-sql-kuldeep.us_bank.transactions
  GROUP BY  account_id
),
customer_level_data AS (
  SELECT    acc_tab.customer_id,
            SUM(acc_tab.balance) AS total_balance,
            SUM(COALESCE(txn_tab.txn_count, 0)) AS total_txn_count
  FROM      scaler-dsml-sql-kuldeep.us_bank.accounts AS acc_tab
  LEFT JOIN count_of_txn AS txn_tab
  ON        acc_tab.account_id = txn_tab.account_id
  GROUP BY  acc_tab.customer_id
)
SELECT      CASE WHEN total_balance > 50000 THEN 'High Balance'
            WHEN total_balance > 10000 THEN 'Medium Balance'
            ELSE 'Low Balance' END AS balance_category,
            COUNT(DISTINCT customer_id) AS customer_count,
            ROUND(AVG(total_balance), 2) AS avg_category_balance,
```

```
            ROUND(AVG(total_txn_count), 2) AS avg_transaction_count,
FROM        customer_level_data
GROUP BY    balance_category
ORDER BY    avg_transaction_count DESC
```

Output

Row	balance_category	customer_count	avg_category_balance	avg_transaction_count
1	High Balance	89	110292.53	1.81
2	Medium Balance	30	33114.65	1.13
3	Low Balance	7	4095.46	0.71

Observation

Activity rises with balance tier: High-balance customers average at 1.81 transactions, Medium at 1.13, Low at 0.71.

Interpretation

The results indicate a **positive relationship between customer balance tier and transaction frequency** in this dataset - customers with higher balances tend to transact more frequently. But the gap between the transaction frequency is not proportional to the gap in count and average balance of the category. So, balance likely is not the only factor influencing the transaction frequency.

Recommendation

US Bank should focus more in strengthening the relationship with the high balance customers by introducing a relationship manager, personalized experiences (understanding their needs and circumstances), transaction-based offerings etc. The Low-balance group is small (7 customers) and can be targeted to convert into Medium-balance customer by probably introducing a starter product, reduced fees etc.

II. B For each account type, find the top 2 months by transaction count in 2025.

```
WITH monthly_txn_count AS (
  SELECT    acc.account_type,
            EXTRACT(MONTH FROM txn.transaction_date) AS txn_month,
            COUNT(*) AS txn_count,
            SUM(txn.amount) AS total_amt
  FROM      scaler-dsml-sql-kuldeep.us_bank.transactions AS txn
  JOIN      scaler-dsml-sql-kuldeep.us_bank.accounts AS acc
  ON        txn.account_id = acc.account_id
  WHERE     EXTRACT(YEAR FROM txn.transaction_date) = 2025
  GROUP BY  txn_month, acc.account_type
),
month_wise_ranking AS (
  SELECT    ROW_NUMBER() OVER(PARTITION BY account_type ORDER BY txn_count DESC) AS
row_num,
-- Used ROW_NUMBER to guarantee exactly 2 months per account type; ties are broken
arbitrarily
            account_type,
            txn_month,
            txn_count,
            total_amt
  FROM      monthly_txn_count
```

```
)
SELECT      account_type,
            txn_month,
            FORMAT_DATE('%B', DATE(2025,txn_month, 15)) AS month_name,
            txn_count,
            total_amt AS total_txn_amt
FROM        month_wise_ranking
WHERE       row_num IN (1, 2)
ORDER BY    account_type, row_num
```

Output

Row	account_type	txn_month	month_name	txn_count	total_txn_amt
1	CHECKING	3	March	3	13687.25
2	CHECKING	4	April	1	8973.11
3	FIXED_DEPOSIT	4	April	3	18265.26
4	FIXED_DEPOSIT	9	September	2	9514.91
5	SAVINGS	6	June	3	14196.39
6	SAVINGS	12	December	2	7019.17

Observation

Transaction volumes are very low across all account types - the busiest months of 2025 for any account type saw only 1 to 3 transactions. That said, the transaction amount and count vary considerably across the given time period and account type.

Interpretation

The sample size seems too small for any meaningful statistical conclusion AS one large transaction can flip the ranking. But again, the top ranked months are scattered which means there's no observable pattern here.

Recommendation

First of all, more data is required for recommending any seasonal offers. But based on the data we have, US Bank should focus more on the FIXED_DEPOSIT offerings AS such account types should have more transaction amount despite low transaction count.

II. C Bucket customers by age (Under 30, 30-50, Over 50) AS of June 1, 2025, then count their credit card transactions.

```
/* Since the transactions table does not contain specific data to relate a transaction to
card_type, in my code I have considered the transactions made by account_id associated
with atleast one CC as CC transactions */
WITH credit_account AS (
  SELECT    DISTINCT
            acc.customer_id,
            acc.account_id,
            age,
            CASE WHEN age < 30 THEN 'Under 30'
            WHEN age between 30 AND 50 THEN '30 - 50'
            WHEN age > 50 THEN 'Over 50'
            ELSE 'Unknown' END AS age_band
```

```sql
-- I have created a separate group for customers with missing date_of_birth
  FROM      scaler-dsml-sql-kuldeep.us_bank.accounts AS acc
  JOIN      (
             SELECT   customer_id,
                      DATE_DIFF(DATE '2025-06-01', date_of_birth, YEAR) -
                         CASE WHEN FORMAT_DATE('%m%d', date_of_birth) >
                         FORMAT_DATE('%m%d', DATE '2025-06-01') THEN 1 ELSE 0 END AS age
       /* DATE_DIFF by year only checks the age in terms of year in the dates. If the
          birthday has not happened yet this year, it will still be considered. So, to
          check the overall date I have used the CASE statement which would subtract 1
          from the age calculated by date_diff if the month and day has not passed yet
          this year */
            FROM     scaler-dsml-sql-kuldeep.us_bank.customers
            ) AS age_table
  ON        acc.customer_id = age_table.customer_id
  JOIN      scaler-dsml-sql-kuldeep.us_bank.cards AS c
  ON        acc.account_id = c.account_id
  WHERE     c.card_type = 'CREDIT'
)
SELECT      ca.age_band,
            COUNT(txn.transaction_id) AS txn_count
FROM        credit_account AS ca
JOIN        scaler-dsml-sql-kuldeep.us_bank.transactions AS txn
ON          ca.account_id = txn.account_id
GROUP BY    ca.age_band
ORDER BY    txn_count DESC
```

Output

Row	age_band	txn_count
1	Over 50	35
2	30 - 50	30
3	Under 30	17
4	Unknown	16

Observation

Credit card transaction activity rises with age: Over 50 customers have the most (35), then 30-50 (30), then Under 30 (17). The 'Unknown' age group (missing date_of_birth) have 16 - almost as many as Under-30.

Interpretation

Older customers may either have more number of cards or may use credit card more for their spending as compared to younger customers. Separately, 16 transactions belonging to customers with no data for date_of_birth seems like data-quality gap.

Recommendation

Investigate the reasons why customers older than 50 tend to transact more via credit card than younger customers and cater the younger customers accordingly so that they transact more using credit cards (like introducing exclusive offers on credit cards). Also, prioritise backfilling the missing date_of_birth to increase the accuracy age-based analysis in the future.

II. D Find the top 10 customers by total balance across their active accounts, including each customer's earliest account opening date.

```
WITH customer_balance AS (
  SELECT    customer_id,
            SUM(balance) OVER(PARTITION BY customer_id) AS total_balance,
            MIN(opened_date) OVER(PARTITION BY customer_id) AS
earliest_account_opening_date,
            COUNT(*) OVER(PARTITION BY customer_id) AS total_number_of_accounts
  FROM      scaler-dsml-sql-kuldeep.us_bank.accounts
  WHERE     status = 'ACTIVE'
)
SELECT      DISTINCT
            cb.customer_id,
            CONCAT(cust.first_name, ' ', cust.last_name) AS customer_name,
            cust.email,
            cb.total_balance,
            cb.total_number_of_accounts,
            cb.earliest_account_opening_date
FROM        scaler-dsml-sql-kuldeep.us_bank.customers AS cust
JOIN        customer_balance AS cb
ON          cb.customer_id = cust.customer_id
ORDER BY    total_balance DESC
LIMIT       10
```

Output

Row	customer_id	customer_name	email	total_balance	total_number_of_accounts	earliest_account_opening_date
1	41	David Patel	smithjulieexample.org	199608.73	3	2021-06-10
2	154	Diana Hebert	brandon39example.net	154251.97	2	2021-12-20
3	94	Beverly Wagner	phyllismeyerexample.net	140071.94	2	2020-03-28
4	151	Joan Johnson	paulproctorexample.org	140069.64	2	2023-08-24
5	90	Justin Marquez	smithjasonexample.org	121257.86	2	2022-06-08
6	105	Loretta Barnes	lindsey69example.org	117072.95	2	2021-02-22
7	12	Rebecca Gardner	rodriguezsierraexample.net	98393.6	1	2023-12-21
8	200	Katelyn Davis	thomaskevinexample.net	96472.08	1	*null*
9	30	Charles Harrington	jsmithexample.com	96289.33	1	*null*
10	11	Eric Campbell	mcmillandennisexample.org	94961.17	1	*null*

Observation

David Patel has the maximum balance (199608.73) as well as maximum number of active accounts (3). Three of the top 10 customers with highest total balance have missing earliest account opening date.

Interpretation

The missing opened_date on some of the bank's highest-valued customers seems like a huge gap in operations. The data also shows that these customers are fairly new (as compared to the earliest account opening date of 2015) so the bank need to find ways to retain these customers.

Recommendation

Prioritize backfilling the opened_date starting with high-balance accounts. And prioritize assigning personal relationship managers and personalized banking experience for the high value customers to retain them.

II. E List customers who currently hold 2 or more active loans, along with their total loan amount.

```sql
WITH active_loans AS (
  SELECT   customer_id,
           ROUND(SUM(loan_amount) OVER(PARTITION BY customer_id), 2) AS total_loan_amount,
           COUNT(*) OVER(PARTITION BY customer_id) AS number_of_active_loans
  FROM     scaler-dsml-sql-kuldeep.us_bank.loans
  WHERE    status = 'ACTIVE'
)
SELECT     DISTINCT
           al.customer_id,
           CONCAT(cust.first_name, ' ', cust.last_name) AS customer_name,
           al.number_of_active_loans,
           al.total_loan_amount
FROM       scaler-dsml-sql-kuldeep.us_bank.customers AS cust
JOIN       active_loans AS al
ON         al.customer_id = cust.customer_id
WHERE      number_of_active_loans > 1
ORDER BY   total_loan_amount DESC
```

Output

Row	customer_id	customer_name	number_of_active_loans	total_loan_amount
1	26	Anthony Perez	2	752963.88
2	57	Shannon Pierce	2	719301.41
3	105	Loretta Barnes	2	533704.88
4	127	Laura Grant	2	454377.42
5	183	Monica Smith	2	367885.54

Observation

The analysis identifies five customers who currently hold more than one active loan. Each of these customers holds exactly 2 active loans. Their combined active loan amounts range from approximately ₹367,886 to ₹752,964. Anthony Perez has the highest total active loan exposure at ₹752,963.88, followed by Shannon Pierce at ₹719,301.41

Interpretation

This group of individuals represent the segment with relatively high loan exposure. But the dataset is too small to make any statistical and significant interpretation. But to better interpret the data needs to be compared with the risk and revenue analysis.

Recommendation

Closely monitor customers holding multiple active loans, particularly those with high total loan amount and consider regular credit-risk reviews, assessment of repayment capacity.

III. A For each year, find the two branches that opened the most accounts.

```sql
WITH account_opened AS (
  SELECT    br.branch_id,
            br.branch_name,
            EXTRACT(YEAR FROM acc.opened_date) AS opened_year,
            COUNT(*)  as number_of_accounts
  FROM      scaler-dsml-sql-kuldeep.us_bank.branches AS br
  JOIN      scaler-dsml-sql-kuldeep.us_bank.accounts AS acc
  ON        br.branch_id = acc.branch_id AND acc.opened_date IS NOT NULL
  GROUP BY  opened_year, branch_id, br.branch_name
),
ranked_table AS (
  SELECT    branch_id,
            branch_name,
            opened_year,
            number_of_accounts,
            ROW_NUMBER() OVER(PARTITION BY opened_year ORDER BY number_of_accounts DESC,
branch_id) AS branch_rank
-- branch_id is used as a deterministic tie-breaker when two branches have the same number
of accounts
  FROM      account_opened
)
SELECT      opened_year,
            branch_name,
            number_of_accounts
FROM        ranked_table
WHERE       branch_rank IN (1, 2)
ORDER BY    opened_year, number_of_accounts DESC
```

Output

Row	opened_year	branch_name	number_of_accounts
1	2018	Brickell Avenue	5
2	2018	West Palm	5
3	2019	Coral Gables	3
4	2019	Hialeah Main	3
5	2020	Downtown Miami	5
6	2020	Brickell Avenue	4
7	2021	Hialeah Main	5
8	2021	Downtown Miami	4
9	2022	Downtown Miami	4
10	2022	West Palm	4

Results per page: 50 1 – 16 of 16

Observation

With the full 16-row result, Downtown Miami appears in the top 2 for number of account openings in 4 of the 8 years (2020, 2021, 2022, 2024) which is half of the entire observed period.

Interpretation

Downtown Miami appears to be a repeated top acquirer of number of new accounts. This is also in line with the data from I. C, that Downtown Miami has the most number of accounts. But the fact that more than 50% of the accounts are in frozen status (from I. C) is also noteworthy.

Recommendation

Study the approach taken by Downtown Miami branch for customer acquisition and use a similar protocol for as a template to test at other branches. Keep an eye on whether the other branches actually acquire more customers.

III. B For each branch, show employee count, account count, total transaction amount, and a performance rank based on transaction amount.

```
WITH employee_count_per_branch AS (
  SELECT      br.branch_id,
              br.branch_name,
              COUNT(emp.employee_id) AS employee_count
  FROM        scaler-dsml-sql-kuldeep.us_bank.branches AS br
  JOIN        scaler-dsml-sql-kuldeep.us_bank.employees AS emp
  ON          br.branch_id = emp.branch_id
  GROUP BY    br.branch_id, br.branch_name
),
account_count_per_branch AS (
  SELECT      br.branch_id,
              br.branch_name,
              COUNT(acc.account_id) AS account_count
  FROM        scaler-dsml-sql-kuldeep.us_bank.branches AS br
  JOIN        scaler-dsml-sql-kuldeep.us_bank.accounts AS acc
  ON          br.branch_id = acc.branch_id
  GROUP BY    br.branch_id, br.branch_name
),
txn_amount_per_branch AS (
  SELECT      acc.branch_id,
              ROUND(SUM(txn.amount), 2) AS total_txn_amount
  FROM        scaler-dsml-sql-kuldeep.us_bank.accounts AS acc
  JOIN        scaler-dsml-sql-kuldeep.us_bank.transactions AS txn
  ON          acc.account_id = txn.account_id
  GROUP BY    acc.branch_id
)
SELECT      br.branch_name,
            COALESCE(emp_count.employee_count, 0) AS employee_count,
            COALESCE(acc_count.account_count, 0) AS account_count,
            COALESCE(txn_amount.total_txn_amount, 0) AS total_txn_amount,
            -- Using COALESCE would replace null values with 0
            DENSE_RANK() OVER (ORDER BY COALESCE(txn_amount.total_txn_amount, 0) DESC) AS
            performance_rank
```

```
FROM        scaler-dsml-sql-kuldeep.us_bank.branches AS br
            -- Using the branches table as the main table and then using left join
            ensures all branches are intact
LEFT JOIN   employee_count_per_branch AS emp_count
ON          br.branch_id = emp_count.branch_id
LEFT JOIN   account_count_per_branch AS acc_count
ON          br.branch_id = acc_count.branch_id
LEFT JOIN   txn_amount_per_branch AS txn_amount
ON          br.branch_id = txn_amount.branch_id
ORDER BY    performance_rank
```

Output

Row	branch_name	employee_count	account_count	total_txn_amount	performance_rank
1	Downtown Miami	14	28	195700.84	1
2	Kendall Branch	20	21	147653.05	2
3	West Palm	19	27	133676.33	3
4	Dade Central	18	20	119072.25	4
5	Brickell Avenue	16	20	116130.26	5
6	South Beach	21	19	96288.95	6
7	Aventura Plaza	10	18	88967.27	7
8	Hialeah Main	15	19	68486.79	8
9	Coral Gables	15	17	61614.41	9
10	Fort Lauderdale	14	5	31726.06	10

Results per page: 50 1 – 12 of 12

Observation

Downtown Miami ranks number 1 in transaction amount (195,700) with only 14 employees which is fewer than several branches ranked below it. South Beach has the most staff (21) but ranks 6th. Observing the last ranked branch - Key Biscayne which has 20 employees (more than Downtown Miami) has only 1 account that too has 0 transaction amount.

Interpretation

Number of employees does not simply mean better performance of the branch. Downtown Miami does better with less employees, while Key Biscayne's larger team is not able to match the performance proportionally.

Recommendation

Study Downtown Miami's operating process and try to replicate the practices of their employees in other branches. Also a detailed review needs to be done for branches where the number of employees is more than the number of accounts and is still not performing as much.

III. C For each (branch, year): total transaction amount, running cumulative total across years, and the yearly average across all branches.

```
WITH combined_tabele AS (
  SELECT    DISTINCT
            brn.branch_name,
            txn.transaction_id,
            txn.amount,
            txn.transaction_date,
            EXTRACT(YEAR FROM transaction_date) AS txn_year
  FROM      scaler-dsml-sql-kuldeep.us_bank.branches AS brn
  JOIN      scaler-dsml-sql-kuldeep.us_bank.accounts AS acc
  ON        brn.branch_id = acc.branch_id
  JOIN      scaler-dsml-sql-kuldeep.us_bank.transactions AS txn
  ON        acc.account_id = txn.account_id
),
yearly_txn_amount AS (
  SELECT    branch_name,
            txn_year,
            ROUND(SUM(amount)) as total_transaction_amount,
  FROM      combined_tabele
  GROUP BY  branch_name, txn_year
  ORDER BY  branch_name, txn_year
)
SELECT      branch_name,
            txn_year,
            total_transaction_amount,
            SUM(total_transaction_amount) OVER(PARTITION BY branch_name ORDER BY txn_year
ROWS BETWEEN UNBOUNDED PRECEDING
            AND CURRENT ROW) AS cumulative_transaction_amount,
            ROUND(AVG(total_transaction_amount) OVER(PARTITION BY txn_year)) AS
avg_transaction_amount_per_year
FROM        yearly_txn_amount
ORDER BY    branch_name, txn_year
```

Output

Row	branch_name	txn_year	total_transaction_amount	cumulative_transaction_amount	avg_transaction_amount_per_year
1	Aventura Plaza	2020	769.0	769.0	17645.0
2	Aventura Plaza	2021	33558.0	34327.0	22621.0
3	Aventura Plaza	2022	4543.0	38870.0	14002.0
4	Aventura Plaza	2023	12254.0	51124.0	19846.0
5	Aventura Plaza	2024	11488.0	62612.0	15753.0
6	Aventura Plaza	2025	26355.0	88967.0	18532.0
7	Brickell Avenue	2020	35948.0	35948.0	17645.0
8	Brickell Avenue	2021	33017.0	68965.0	22621.0
9	Brickell Avenue	2022	24710.0	93675.0	14002.0
10	Brickell Avenue	2023	12383.0	106058.0	19846.0

Results per page: 50 1 – 50 of 59

Observation

The full result shows that Downtown Miami is the top branch by cumulative transaction total (195,701), matching III.B's figure almost exactly (confirms that observation). It also shows North Miami has only one year of data (2022, and just 5,274), while Fort Lauderdale is missing the data for 2024 entirely but has the all the other year data from 2020 to 2025.

Interpretation

Based on the data, North Miami likely either opened very recently or is barely active while Fort Lauderdale's missing 2024 suggests a serious operational gap specific year. Also, Downtown Miami's YoY performance variation is proportionally the small compared to any other major branch. It's not just the biggest branch in terms of number but is also the top performer in almost all parameters.

Recommendation

Confirm directly with branch operations if North Miami is a new branch or is it an operational gap. For Fort Lauderdale a review needs to be done to check the operational gap which led to lack of data in 2024. Separately, Downtown Miami's performance in both the scale and stability makes it the strongest branch to set an example of operating model (confirmed with the III.B and III.A findings).

III. D Find the total loan amount issued, active loans count, average interest rate, and % of loans paid, per branch.

```sql
WITH active_loan_count AS (
  SELECT        branch_id,
                COUNT(loan_id) as number_of_active_loan
  FROM          scaler-dsml-sql-kuldeep.us_bank.loans
  WHERE         status = 'ACTIVE'
  GROUP BY      branch_id
),
loan_data AS (
/*ASSUMPTION: Computation across ALL loans regardless of status*/
  SELECT        b.branch_id,
                b.branch_name,
                ROUND(SUM(l.loan_amount), 2) AS total_loan_amount,
                ROUND(AVG(l.interest_rate), 2) AS avg_interest_rate,
                ROUND(SUM(CASE WHEN status = 'PAID' THEN 1 ELSE 0 END) * 100.0 /
                   COUNT(l.loan_id), 2) AS percentage_of_loans_paid
  FROM          scaler-dsml-sql-kuldeep.us_bank.branches AS b
  LEFT JOIN     scaler-dsml-sql-kuldeep.us_bank.loans AS l
  ON            b.branch_id = l.branch_id
-- LEFT JOIN keeps branches with zero loans in the output
  GROUP BY      b.branch_id, b.branch_name
)
SELECT          ld.branch_id,
                ld.branch_name,
                IFNULL(ld.total_loan_amount, 0) AS total_loan_amount,
                IFNULL(alc.number_of_active_loan, 0) AS number_of_active_loan,
                IFNULL(ld.avg_interest_rate, 0) AS avg_interest_rate,
                IFNULL(ld.percentage_of_loans_paid, 0) AS percentage_of_loans_paid
FROM            loan_data AS ld
LEFT JOIN       active_loan_count AS alc
ON              ld.branch_id = alc.branch_id
ORDER BY        total_loan_amount DESC
```

Output

Row	branch_id	branch_name	total_loan_amount	number_of_active_loan	avg_interest_rate	percentage_of_loans_paid
1	4	Kendall Branch	6933541.74	7	8.12	45.83
2	9	West Palm	6347648.03	11	9.04	25.0
3	5	Dade Central	5899474.11	8	9.81	23.81
4	1	Downtown Miami	5777293.87	8	9.27	29.17
5	7	Hialeah Main	5665030.22	7	8.77	35.0
6	2	South Beach	5445106.45	6	8.28	36.84
7	6	Brickell Avenue	5361644.51	2	9.08	40.0
8	8	Aventura Plaza	4553607.41	5	8.84	45.0
9	3	Coral Gables	4489395.51	9	8.09	26.32
10	12	North Miami	1774444.92	1	8.43	60.0

Results per page: 50 1 – 12 of 12

Observation

Loan-paid rates vary from 23.8% to 60%. North Miami has the highest percentage of loans paid but has only one active loan. West Palm and Dade Central have some of the largest number of active loans and total loan amount but it's percentage of loans paid is very low.

Interpretation

North Miami's 60% doesn't simply mean that its performing good because of the very small sample size. But West Palm's data is the most credible among all. Kendall Branch is doing good with the highest total loan amount and a good percentage of loans paid.

Recommendation

Reviewing West Palm and Dade Central's loan collections process specifically.

IV. A For each loan type (PERSONAL, MORTGAGE, AUTO, BUSINESS), find the three branches with the highest default rate.

```
WITH branch_total_loans AS (
  -- ALL loans regardless of status, per (branch, loan_type)
  SELECT    branch_id,
            loan_type,
            COUNT(loan_id) AS loan_count
  FROM      scaler-dsml-sql-kuldeep.us_bank.loans
  GROUP BY  branch_id, loan_type
),
branch_default_loans AS (
  -- Defaulted loans per (branch, loan_type)
  SELECT    branch_id,
            loan_type,
            COUNT(loan_id) AS defaulted_loans
  FROM      scaler-dsml-sql-kuldeep.us_bank.loans
  WHERE     status = 'DEFAULTED'
  GROUP BY  branch_id, loan_type
),
```

```sql
ranked_list AS (
  -- Ranking on the basis of loan_type and default_rate_percentage
  SELECT    bdl.branch_id,
            bdl.loan_type,
            btl.loan_count,
            bdl.defaulted_loans,
            ROUND((bdl.defaulted_loans/btl.loan_count)*100, 1) AS default_rate_percentage,
            DENSE_RANK() OVER(PARTITION BY bdl.loan_type ORDER BY
            ROUND((bdl.defaulted_loans/btl.loan_count)*100, 2) DESC) AS rnk
/* ASSUMPTION: DENSE_RANK chosen over ROW_NUMBER so branches tied at the same
   rate all appear, rather than arbitrarily keeping only one. */
  FROM      branch_default_loans AS bdl
  JOIN      branch_total_loans AS btl
  ON        bdl.branch_id = btl.branch_id AND bdl.loan_type = btl.loan_type
)
SELECT      br.branch_id,
            rl.loan_type,
            br.branch_name,
            rl.loan_count,
            rl.defaulted_loans,
            rl.default_rate_percentage
FROM        ranked_list AS rl
JOIN        scaler-dsml-sql-kuldeep.us_bank.branches AS br
ON          rl.branch_id = br.branch_id
WHERE       rnk IN (1, 2, 3)
ORDER BY    rl.loan_type, rl.rnk
```

Output

Row	branch_id	loan_type	branch_name	loan_count	defaulted_loans	default_rate_percentage
1	6	AUTO	Brickell Avenue	5	3	60.0
2	1	AUTO	Downtown Miami	6	3	50.0
3	5	AUTO	Dade Central	2	1	50.0
4	8	AUTO	Aventura Plaza	4	2	50.0
5	12	AUTO	North Miami	2	1	50.0
6	2	AUTO	South Beach	8	2	25.0
7	10	BUSINESS	Fort Lauderdale	2	2	100.0
8	7	BUSINESS	Hialeah Main	6	4	66.7
9	9	BUSINESS	West Palm	8	5	62.5
10	11	MORTGAGE	Key Biscayne	1	1	100.0

Results per page: 50 1 – 16 of 16

Observation

The full 16-row result confirms some loan types are risky with over 60% default rate.

BUSINESS loans - Fort Lauderdale 100% of 2 loans, Hialeah Main 66.7% of 6 loans, West Palm 62.5% of 8 loans.

MORTGAGE loans - Key Biscayne 100% of 1 loan, , Coral Gables 66.7% of 3 loans, and Kendall Branch 62.5% of 8 loans.

Interpretation

Two loan types - BUSINESS and MORTGAGE show a high-default rates (inclusive of multiple branches). Notably, the data for West Palm is also confirmed from the low loan-paid rate in III.D, and the high default rate in West Palm, Kendall Branch could be also due to both being largest total loan book (seen in III. D). The data of loan volume and default rates together acts as a more concrete evidence of the risk factor for the loan-type.

Recommendation

West Palm and Kendall Branch should be treated as priority branches for a direct servicing review because both are flagged on the report form III. D and IV. A. Both West Palm and Kendall Branch have high loan volume with elevated default rates across more than one metric in this report, making the finding a stronger evidence.

IV. B For each branch, find the total interest revenue from active loans, and its % contribution to the bank-wide total.

```sql
WITH branch_loans AS  (
  --Total interest revenue from ACTIVE loans only, per branch.
  SELECT      branch_id,
              ROUND(SUM((loan_amount*interest_rate)/100), 2) AS interest_revenue
  FROM        scaler-dsml-sql-kuldeep.us_bank.loans
  WHERE       status = 'ACTIVE'
  GROUP BY    branch_id
)
SELECT        br.branch_id,
              br.branch_name,
              IFNULL(bl.interest_revenue, 0) AS branch_interest_revenue,
              -- Branches with zero ACTIVE loans are shown with interest_revenue = 0
              ROUND((IFNULL(bl.interest_revenue, 0) /
              IFNULL(SUM(interest_revenue) OVER(), 0))*100, 2) AS
branch_percentage_contribution
FROM          scaler-dsml-sql-kuldeep.us_bank.branches AS br
LEFT JOIN     branch_loans AS bl
-- LEFT JOIN Keeps all brnach names from brnahces table
ON            br.branch_id = bl.branch_id
ORDER BY      branch_interest_revenue DESC
```

Output

Row	branch_id	branch_name	branch_interest_revenue	branch_percentage_contribution
1	1	Downtown Miami	225664.13	16.12
2	9	West Palm	213951.32	15.28
3	5	Dade Central	206676.99	14.76
4	3	Coral Gables	175736.69	12.55
5	4	Kendall Branch	148500.64	10.61
6	7	Hialeah Main	139581.82	9.97
7	2	South Beach	120944.24	8.64
8	8	Aventura Plaza	108872.26	7.78
9	6	Brickell Avenue	44748.11	3.2

Results per page: 50 1 – 12 of 12

Observation

The top 3 branches (Downtown Miami, West Palm, Dade Central) together generate 46.2% of total active-loan interest revenue.

Interpretation

The disproportionate distribution of interest revenue from branch means that a disruption at any one of the top three would hit interest income very hard. Yet again Downtown Miami appears to be the flagship branch as seen before. Even though West Palm has high revenue from interest but the data from previous reports also suggest that West Palm has low percentage of loans paid and high default rate.

Recommendation

Prioritize Downtown Miami, West Palm, Dade Central for disruption free operations by ensuring backup staffs, reliable security systems, and strong fraud monitoring systems. At the same time, look at what is driving their higher loan volumes and see whether those practices can be applied to smaller branches.

IV. C Find active loans with either no payments at all, or payments overdue by more than 180 days, as of April 30, 2025.

```sql
WITH loan_details AS (
/* Pre-aggregate to one row per loan_id BEFORE joining to loans, so a
loan with zero payments simply produces no row here. */
  SELECT      l.loan_id,
              l.customer_id,
              lp.payment_date,
/* payment_date column in the CSV files has dates in different format for different rows.
But BigQuery adjusts them to the same format, so I need not format the date */
              l.loan_amount,
              MAX(lp.payment_date) OVER(PARTITION BY l.loan_id) AS last_payment_date,
              TIMESTAMP_DIFF('2025-04-30', MAX(lp.payment_date) OVER(PARTITION BY
l.loan_id), DAY)
              AS days_since_last_payment
  FROM        scaler-dsml-sql-kuldeep.us_bank.loans AS l
  LEFT JOIN   scaler-dsml-sql-kuldeep.us_bank.loan_payments AS lp
  ON          l.loan_id = lp.loan_id AND lp.payment_date <= '2025-04-30'
  -- LEFT JOIN preserves the loans with no payements at all
  WHERE       l.status = 'ACTIVE'
)
SELECT        DISTINCT
              ld.loan_id,
              ld.customer_id,
              CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
              ROUND(ld.loan_amount, 2) AS loan_amount,
              ld.days_since_last_payment
FROM          loan_details AS ld
JOIN          scaler-dsml-sql-kuldeep.us_bank.customers AS c
ON            ld.customer_id = c.customer_id
-- Never-paid loans (NULL last_payment_date) are included unconditionally
WHERE         ld.days_since_last_payment IS NULL OR ld.days_since_last_payment > 180
ORDER BY      ld.days_since_last_payment DESC NULLS FIRST
-- NULL days_since_last_payment are shown first
```

Output

Row	loan_id	customer_id	customer_name	loan_amount	days_since_last_payment
1	79	19	Johnny Miller	76711.78	*null*
2	189	95	Margaret Terry	231378.31	*null*
3	6	104	Jackson Mcclure	400495.66	*null*
4	188	115	Jaclyn Barnes	267471.56	*null*
5	168	131	Gregory Obrien	453055.44	*null*
6	155	103	Heidi Bowen	71795.45	*null*
7	145	26	Anthony Perez	426543.58	*null*
8	26	105	Loretta Barnes	462956.15	*null*
9	137	85	Courtney Brooks	394533.23	*null*
10	39	44	Lisa Monroe	142609.42	*null*

Results per page: 50 1 – 50 of 55

Observation

All 10 most severely overdue active loans shown have zero payments ever, not just overdue, and this amounts up to 462,956. Customer 105 (Loretta Barnes) appears here and also shows up in II. D as having one of the highest total balance and in II. E for having one of the highest total loan amount (being a multi-loan holder).

Interpretation

Zero payments ever could either mean a recently disbursed loan, or it can be genuine non-repayment of the loan. Loretta Barnes specifically is a case where high deposit value and loan risk sit on the same customer.

Recommendation

First this data needs to be combined with the loan disbursement date and on the basis of that we can categorize the genuine non-repayment as urgent matter. Customers like Loretta Barnes (having one of the highest total balance and one of the highest total loan amount) should be assigned a relationship-manager to review rather than routine collections.

IV. D For each year, find the Jan-Aug transaction total, the previous year's Jan-Aug total, and the % increase. Exclude the first year (no prior year to compare against).

```
WITH month_wise_date AS (
  SELECT      DISTINCT
              FORMAT_DATE('%Y-%m', transaction_date) AS month_year,
              EXTRACT(MONTH FROM transaction_date) AS month_number,
              EXTRACT(YEAR FROM transaction_date) AS year_number,
              SUM(amount) OVER(PARTITION BY FORMAT_DATE('%Y-%m', transaction_date)) AS
              total_amount_per_month
  FROM        scaler-dsml-sql-kuldeep.us_bank.transactions
  WHERE       EXTRACT(MONTH FROM transaction_date) BETWEEN 1 AND 8
  ORDER BY    month_year
),
yearly_data AS (
  SELECT      year_number,
              ROUND(SUM(total_amount_per_month), 2) AS year_jan_to_aug_amount
  FROM        month_wise_date
  GROUP BY    year_number
```

```
)
SELECT        year_number AS year,
              year_jan_to_aug_amount AS total_transaction_amount,
              LAG(year_jan_to_aug_amount) OVER(ORDER BY year_number) AS
              previous_years_total_amount,
--ASSUMPTION: the earliest year in the data intentionally shows NULL here
              ROUND((year_jan_to_aug_amount-LAG(year_jan_to_aug_amount)
              OVER(ORDER BY year_number))/
              LAG(year_jan_to_aug_amount) OVER(ORDER BY year_number)*100, 2)
              AS percentage_increaase_in_amount
--ASSUMPTION: the earliest year in the data intentionally shows NULL here
FROM          yearly_data
ORDER BY      year_number
```

Output

Row	year	total_transaction_amount	previous_years_total_amount	percentage_increaase_in_amount
1	2020	123536.86	*null*	*null*
2	2021	166949.56	123536.86	35.14
3	2022	123463.67	166949.56	-26.05
4	2023	137243.18	123463.67	11.16
5	2024	96727.81	137243.18	-29.52
6	2025	136672.03	96727.81	41.3

Observation

Transaction amounts from January to August show significant year-to-year variation. (Jan-Aug): +35%, -26%, +11%, -30%, +41%.

Interpretation

This data either looks like noise, not a real trend or the trend is genuinely volatile rather than consistently increasing.

Recommendation

If the data actually turns out to be genuinely volatile then a proper investigation should be done to identify the factors behind the sharp declines in 2022 and 2024 and identify what contributed to the strong recovery in 2025.

V. A Per branch: total interactions, complaint count, complaint rate %, employee count, and the list of employee positions.

```
WITH branch_details AS (
    SELECT    DISTINCT
              br.branch_id,
              br.branch_name,
              COUNT(ci.interaction_id) AS total_number_of_interactions,
              COUNT(DISTINCT emp.employee_id) AS total_number_of_employees,
              SUM(CASE WHEN ci.interaction_type = 'COMPLAINT' THEN 1 ELSE 0 END) AS
number_of_complaint,
              STRING_AGG(DISTINCT emp.position, ', ' ORDER BY emp.position ASC) AS
employee_positions
```

```
  FROM        scaler-dsml-sql-kuldeep.us_bank.branches AS br
  LEFT JOIN   scaler-dsml-sql-kuldeep.us_bank.employees AS emp
  ON          br.branch_id = emp.branch_id
  LEFT JOIN   scaler-dsml-sql-kuldeep.us_bank.customer_interactions AS ci
  ON          emp.employee_id = ci.employee_id
  GROUP BY    br.branch_id, br.branch_name
)
SELECT        branch_name,
              total_number_of_interactions,
              total_number_of_employees,
              IFNULL(number_of_complaint, 0) as number_of_complaints,
              ROUND((IFNULL(number_of_complaint, 0)/total_number_of_interactions)*100, 2)
              AS complaint_rate_percentage,
              employee_positions,
FROM          branch_details
ORDER BY      complaint_rate_percentage DESC
```

Output

Row	branch_name	total_number_of_interactions	total_number_of_employees	number_of_complaints	complaint_rate_percentage	employee_positions
1	Brickell Avenue	13	16	6	46.15	Accountant, Compliance Officer, Customer Service Representative, Mortgage Specialist, Teller
2	Hialeah Main	11	15	4	36.36	Accountant, Branch Manager, Compliance Officer, Customer Service Representative, Financial Advisor, IT Support Specialist, Loan Officer, Mortgage Specialist, Operations Manager
3	South Beach	28	21	10	35.71	Accountant, Branch Manager, Compliance Officer, Customer Service Representative, IT Support Specialist, Loan Officer, Operations Manager, Teller
4	Downtown Miami	20	14	7	35.0	Accountant, Compliance Officer, IT Support Specialist, Mortgage Specialist, Operations Manager, Teller
5	Coral Gables	10	15	3	30.0	Branch Manager, Compliance Officer, Customer Service Representative, Financial Advisor, IT Support Specialist, Loan Officer, Operations Manager
6	Kendall Branch	26	20	7	26.92	Accountant, Branch Manager, Compliance Officer, Customer Service Representative, Financial Advisor, IT Support Specialist, Mortgage Specialist, Operations Manager, Teller
7	Aventura Plaza	13	10	3	23.08	Accountant, Compliance Officer, Financial Advisor, IT Support Specialist
8	Fort Lauderdale	13	14	3	23.08	Accountant, Branch Manager, Customer Service Representative, Financial Advisor, Mortgage Specialist, Teller
9	Key Biscayne	18	20	4	22.22	Accountant, Branch Manager, Compliance Officer, Financial Advisor, IT Support Specialist, Loan Officer, Mortgage Specialist, Operations Manager
10	Dade Central	11	18	2	18.18	Accountant, Branch Manager, Compliance Officer, Customer Service Representative, IT Support Specialist, Loan Officer, Mortgage Specialist, Operations Manager, Teller

Results per page: 50 1 – 12 of 12

Observation

Brickell Avenue has the highest complaint rate (46.15%, 6 of 13 interactions), but South Beach has the highest complaint volume (10 complaints).

Interpretation

Brickell Avenue’s issue is more severe (relative to its size) while South Beach's is more about overall volume/capacity. North Miami has the least complaint rate but that could also be because we have only one year data for this branch (as established in III. C)

Recommendation

Investigate Brickell Avenue for a specific root cause of such high complaint rate. Also consider increasing employee count at South Beach given its much higher interaction load.

V. B Find customers who complained in 3 or more distinct months (not 3 total complaints -- 3 different months).

```sql
WITH complaints_cte AS (
    SELECT  interaction_id,
            customer_id,
            interaction_date,
            FORMAT_DATE('%Y-%m', interaction_date) AS year_month,
            FORMAT_DATE('%B %Y', interaction_date) AS month_name
  FROM      scaler-dsml-sql-kuldeep.us_bank.customer_interactions
  WHERE     interaction_type = 'COMPLAINT'
),
customer_month AS (
  SELECT    DISTINCT
            customer_id,
            year_month,
            month_name
  FROM      complaints_cte
),
customer_complaints AS (
  SELECT
            customer_id,
            COUNT(*) AS total_complaints,
            COUNT(DISTINCT year_month) AS number_of_complaint_months
  FROM      complaints_cte
  GROUP BY  customer_id
  HAVING    COUNT(DISTINCT year_month) >= 3
)
SELECT    cc.customer_id,
          CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
          cc.number_of_complaint_months,
          cc.total_complaints,
          STRING_AGG(cm.month_name, ', ' ORDER BY cm.year_month) AS complaint_months
-- Sorted chronologically via year_month, NOT alphabetically by month_name
FROM      customer_complaints AS cc
JOIN      customer_month AS cm
ON        cc.customer_id = cm.customer_id
JOIN      scaler-dsml-sql-kuldeep.us_bank.customers AS c
ON        cc.customer_id = c.customer_id
GROUP BY  cc.customer_id,
          c.first_name,
          c.last_name,
          cc.number_of_complaint_months,
          cc.total_complaints

ORDER BY  cc.number_of_complaint_months DESC,
          cc.total_complaints DESC;
```

Output

Row	customer_id	customer_name	number_of_complaint_months	total_complaints	complaint_months
1	198	Nathan Fernandez	3	3	October 2020, May 2021, June 2024

Observation

Only one customer (customer_id 198) in the whole dataset meets the 3+ distinct-month threshold. The complaints are spread years apart rather than clustered together.

Interpretation

Complaints being year apart suggests that complaints are not a wide-spread problem but are one-off incidents. Also, the long gap means that complaints are for one single ongoing unresolved issue.

Recommendation

With only one customer qualifying the criteria, there needs to be a direct account review rather than a change in system and policy.

VI. Dataset limitation:

The schema links cards and transactions through account_id, but transactions do not contain a card_id. As a result, account-level balances cannot always be uniquely attributed to an individual card when an account has multiple cards. This may contribute to unusually high or potentially misleading card-level credit-utilization percentages.

VI. A Find active credit card holders whose account balance is 90%+ of their credit limit.

```sql
SELECT      cu.customer_id,
            CONCAT(cu.first_name, ' ', cu.last_name) AS customer_name,
            CAST(c.card_number AS STRING) AS card_number,
-- In BigQuery the datatype for card_number was float, so, had to convert it to string
            c.credit_limit,
            acc.balance,
            ROUND((acc.balance*100)/c.credit_limit, 2) AS credit_utilization
FROM        scaler-dsml-sql-kuldeep.us_bank.customers AS cu
JOIN        scaler-dsml-sql-kuldeep.us_bank.accounts AS acc
ON          cu.customer_id = acc.customer_id
JOIN        scaler-dsml-sql-kuldeep.us_bank.cards AS c
ON          acc.account_id = c.account_id
WHERE       c.card_type = 'CREDIT' AND
            c.card_status = 'ACTIVE' AND
            c.credit_limit IS NOT NULL AND
            acc.balance >= (0.9*c.credit_limit)
```

Output

Row	customer_id	customer_name	card_number	credit_limit	balance	credit_utilization
1	146	Abigail Cole	570541152627	4867.61	43915.34	902.2
2	164	Amanda Ball	30464887719062	8810.94	69031.35	783.47
3	179	Amy Bates	4458685014294019	8588.61	55441.25	645.52
4	182	Cassandra Kent	4134936183242102	9986.49	43548.71	436.08
5	78	Donald Turner	180023924075819	16590.19	54793.72	330.28
6	114	Frederick Meadows	213125871397186	14958.05	20355.82	136.09
7	83	Gary Cunningham	3570866882634353	2303.75	98077.46	4257.3
8	83	Gary Cunningham	30437489691087	13610.09	98077.46	720.62
9	83	Gary Cunningham	4752500743292717	14161.39	98077.46	692.57
10	170	Jennifer Jacobs	3599727875588674	17428.36	48667.14	279.24

Results per page: 50 1 – 22 of 22

Observation

Several customers have account balances that significantly exceed their stated credit limits. For example, Gary Cunningham has an account balance of 98,077.46 across three active credit cards, while the individual credit limits are only 2,303.75, 13,610.09, and 14,161.39. Due to this his credit utilization ranges from 692.57% to 4,257.30%.

Other customers also show utilization well above 100%, such as Abigail Cole at 902.2% and Amanda Ball at 783.47%.

Interpretation

The repeated identical balance hints that all 3 cards are linked to the same account, so balance here means money in that account, not amount used using credit card. This does not seem like a case of extreme overspending, it's one dataset gap.

Recommendation

Before acting on this metric as a potential risk signal, first a column of card_id needs to be added in transactions table to get the actual data of credit utilization. And finally, a targeted credit-risk assessments needs to be conducted for genuinely over-utilized accounts.

VI. B For each (card_type, month in 2025): total spend, unique cardholders, average transaction size -- for active cards only.

```sql
WITH active_card_accounts AS (
    /* active_card_accounts deduplicates rows when a single account_id holding
    two or more cards of the same type.
    REMAINING DATA LIMITATION: accounts holding cards of DIFFERENT types still
    produce distinct rows here. */
    SELECT  DISTINCT
            account_id,
            card_type
    FROM    scaler-dsml-sql-kuldeep.us_bank.cards
    WHERE   card_status = 'ACTIVE'
)
SELECT      aca.card_type,
            EXTRACT(MONTH FROM t.transaction_date) AS month,
            ROUND(SUM(t.amount), 2) AS total_amount_spent,
            COUNT(DISTINCT a.customer_id) AS unique_cardholders,
            ROUND(AVG(t.amount), 2) AS average_transaction_size
FROM        active_card_accounts AS aca
JOIN        scaler-dsml-sql-kuldeep.us_bank.accounts AS a
ON          aca.account_id = a.account_id
JOIN        scaler-dsml-sql-kuldeep.us_bank.transactions AS t
ON          a.account_id = t.account_id
```

```
WHERE       EXTRACT(YEAR FROM t.transaction_date) = 2025
GROUP BY    aca.card_type, month
ORDER BY    aca.card_type, month
```

Output

Row	card_type	month	total_amount_spent	unique_cardholders	average_transaction_size
1	CREDIT	3	3754.66	1	3754.66
2	CREDIT	4	8973.11	1	8973.11
3	CREDIT	6	5430.69	1	5430.69
4	DEBIT	1	9662.33	1	9662.33
5	DEBIT	2	6909.19	1	6909.19
6	DEBIT	3	6951.1	2	3475.55
7	DEBIT	5	5777.75	1	5777.75
8	DEBIT	8	11302.71	2	5651.35
9	DEBIT	9	9514.91	2	4757.45
10	DEBIT	10	5171.11	1	5171.11

Results per page: 50 1 – 11 of 11

Observation

The full result has only 11 rows, which means that all the possible combinations (12 months x 2 card types = 24) are not seen in the output. This means that CREDIT card spend only appears in 3 of 12 months in 2025 (March, April, June), while DEBIT card spend appears in 8 of 12. Every single month and card-type combination has just 1-2 unique cardholders.

Interpretation

According to the output, active credit card spending is genuinely less and sporadic, while debit card usage is far more consistent from month to month. And the small cardholder counts for all combinations also hint that active card spending looks either very early-stage or very low-adoption.

Recommendation

Need to investigate the huge gap in usage of credit and debit card. This seems like a gap in the awareness of using credit and debit cards. Customers need to be targeted accordingly to adopt to using credit card.

VI. C For each (card_type, transaction_type) combo: count, total amount, average amount.

```
WITH active_card_accounts AS (
    /* active_card_accounts deduplicates rows when a single account_id holding
    two or more cards of the same type.
    REMAINING DATA LIMITATION: accounts holding cards of DIFFERENT types still
    produce distinct rows here. */
    SELECT  DISTINCT
            account_id,
            card_type
    FROM    scaler-dsml-sql-kuldeep.us_bank.cards
)
SELECT
            aca.card_type,
            t.transaction_type,
            COUNT(t.transaction_id) AS number_of_transactions,
            ROUND(SUM(t.amount), 2) AS total_transaction_amount,
            ROUND(AVG(t.amount), 2) AS average_transaction_amount
FROM        active_card_accounts AS aca
JOIN        scaler-dsml-sql-kuldeep.us_bank.transactions AS t
```

```
ON          aca.account_id = t.account_id
 --Need not join accounts table because account_id is already present in
active_card_accounts
GROUP BY    aca.card_type, t.transaction_type
ORDER BY    card_type, number_of_transactions DESC
```

Output

Row	card_type	transaction_type	number_of_transactions	total_transaction_amount	average_transaction_amount
1	CREDIT	TRANSFER	35	201717.32	5763.35
2	CREDIT	PAYMENT	22	121883.22	5540.15
3	CREDIT	WITHDRAWAL	21	140174.4	6674.97
4	CREDIT	DEPOSIT	20	89207.76	4460.39
5	DEBIT	TRANSFER	30	145922.27	4864.08
6	DEBIT	PAYMENT	17	97130.5	5713.56
7	DEBIT	WITHDRAWAL	17	81839.88	4814.11
8	DEBIT	DEPOSIT	13	81344.0	6257.23

Observation

CREDIT cards shows higher transaction counts than DEBIT across every transaction type. Transaction type TRANSFER via CREDIT card shows the highest number of total transactions. But the highest average transaction amount is seen in DEPOSIT via DEBIT card.

Interpretation

The result shows that transaction_type TRANSFER has the most number of transactions and total_transaction_amount across both card types while DEPOSIT being the least in both. Also the data consistently shows that for every transaction_type, customers prefer CREDIT card over DEBIT card.

Recommendation

Monitoring the WITHDRAWAL using CREDIT card is important as it constitutes the highest average_transaction_amount across both card types.

Overall Recommendations

Throughout the project, a few trends have been across multiple segments and are evidenced in more than one independent query. These are worth noticing as bank-wide finding rather than isolated occurring.

1. Downtown Miami is clearly the flagship branch. It is the leading branch in terms of total transaction volume with proportionally less number of employees. It has been in the top 2 branch four times in the last year in terms of count of new account opened. It also has the most stable year-over-year performance of any major branch despite being the largest. These positive signals point towards an operational model which is genuinely worth replicating in all other branches. That said, the only concern is the large proportion of frozen accounts which is worth investigating because this branch matters so much to the network.

2. The pattern clearly points that West Palm and Kendall Branch are two branches with the most loan risk. West Palm shows a low loan-paid rate on real volume and a high BUSINESS-loan default rate as well. Kendall Branch holds the single largest total loan book in the network of the bank also showing a high MORTGAGE-loan default rate. The same two branches surface across two independent questions each and needs to be investigated further to resolve these issues.

3. The single biggest structural gap in the dataset is the missing card_id on transactions. This affects determining several metrics like complete Card Portfolio independently.

4. Revenue across the bank is concentrated in the top 3 branches which generate 46.2% of interest revenue. This makes it mandatory to plan the continuous smooth planning in these branches rather than just celebrating their output.

5. A small number of customers recur across unrelated segments like Loretta Barnes deserves a relationship-manager-level review. And several findings based on small samples produced dramatic but low-confidence numbers.